TCP/IP suite Theory

Task 1: Wireshark Capture on Flipkart & Header Layer Identification

How to Capture:

1. Open **Wireshark**, select your active connection (e.g., **Wi-Fi** or **Ethernet**), and click Start.
2. Open a browser, navigate to **flipkart.com**, and refresh the page.
3. Stop the capture in Wireshark and expand the packet header details in the middle inspection window.

Identified Packets Across the 4 TCP/IP Layers:

TCP/IP Layer	Identified Protocol / Packet	Observed Header Details in Wireshark	How It Was Identified
1. Application Layer	**HTTP / TLS / DNS**	Transport Layer Security -> TLSv1.3 Record Layer: Encrypted Application Data	Found under the application payload carrying the web page assets, images, and user session requests.
2. Transport Layer	**TCP**	Transmission Control Protocol, Src Port: 53124, Dst Port: 443 [ACK] Seq=1 Ack=1	Identified by the TCP header showing Source/Destination ports (443 for HTTPS) and sequence/acknowledgment numbers.
3. Internet Layer	**IPv4 (Internet Protocol)**	Internet Protocol Version 4, Src: 192.168.1.15, Dst: 163.53.76.86	Identified by the IP header listing the client local IP and Flipkart's edge CDN server IP.
4. Network Access Layer	**Ethernet II / Wi-Fi**	Ethernet II, Src: Intel_xx:xx:xx (Your MAC), Dst: Tp-Link_yy:yy:yy (Router MAC)	Identified by the physical hardware MAC address header linking your PC directly to the local default gateway.

Day 2: TCP/IP Encapsulation & Message Transmission Flow

Task 4: Step-by-Step Data Flow from Your Device (Client) to Instagram Cloud Server across 4 TCP/IP Layers

1. APPLICATION LAYER

PDU: Application Data (Payload)

Action: Message Creation & Formatting

App-Level Step: You type 'Hey!' inside an Instagram Direct Message (DM) and hit send.

Technical Action: The Instagram client packages text and user authentication tokens into an HTTPS / TLS request payload.

Encapsulation State: **+ Application Data Payload**

2. TRANSPORT LAYER

PDU: TCP Segment

Action: Message Segmentation & Port Address

App-Level Step: Large files (photos/reels) or messages are split into numbered segments so nothing gets lost.

Technical Action: Adds TCP Header: Source Port 54321, Destination Port 443 (HTTPS), and Sequence Numbers for reliability.

Encapsulation State: **[TCP Header (Ports 443/54321, Seq #)] + [Data]**

3. INTERNET LAYER

PDU: IP Packet

Action: Logical Addressing & Routing Envelope

App-Level Step: The system attaches 'postal addresses' so routers know where the message came from and where it is going.

Technical Action: Adds IP Header: Source IP (e.g., 192.168.1.15) and Destination IP (Meta Server: 157.240.22.35).

Encapsulation State: **[IP Header (Source & Dest IP)] + [TCP Header] + [Data]**

4. NETWORK ACCESS LAYER

PDU: Frame & Raw Bits

Action: Local MAC Framing & Physical Transmi

App-Level Step: Data is transformed into physical Wi-Fi radio waves or electrical/optical pulses leaving your device.

Technical Action: Adds Ethernet/Wi-Fi Header (Source & Router Gateway MAC) + Trailer (FCS/CRC error checking). Transmitted as bits.

Encapsulation State: **[MAC Header] + [IP Header] + [TCP Header] + [Data] + [CRC Check]**

RECEPTION & DE-ENCAPSULATION AT INSTAGRAM SERVER (Reverse Process):

1. Network Access: Receives physical bitstream and checks MAC frame integrity -> strips MAC header.
2. Internet Layer: Inspects destination IP -> verifies it matches Instagram server -> strips IP header.
3. Transport & Application: Reassembles TCP segments in sequence, decodes TLS, and updates the recipient's chat inbox.

Task 2: Comparison Table: OSI Model vs. TCP/IP Model with Real-World App Examples

OSI Model (7 Layers) TCP/IP Model (4 Layers) Protocols & PDU Real-World Everyday App Example



Task 3: TCP Segment Header Fields & Explanations

In the Transport layer, a TCP header consists of the following standard fields:

1. **Source Port (16 bits):**

Identifies the sending application/process on your local device (e.g., dynamically assigned port 54210).

2. **Destination Port (16 bits):**

Identifies the target service on the destination server (e.g., Port 443 for HTTPS or Port 80 for HTTP).

3. **Sequence Number (32 bits):**

Assigns an incremental number to each byte sent so the receiver can reassemble split packets in the exact original order.

4. **Acknowledgment Number (32 bits):**

Confirms receipt of data by indicating the next expected sequence number the receiver anticipates.

5. **Data Offset / Header Length (4 bits):**

Specifies the total size of the TCP header in 32-bit words, indicating where the actual payload begins.

6. **Control Flags (9 bits - SYN, ACK, FIN, RST, PSH, URG):**

Controls connection management (e.g., SYN initiates the 3-way handshake, ACK acknowledges receipt, FIN closes the connection).

7. **Window Size (16 bits):**

Manages flow control by indicating how many bytes the receiving device's buffer can accept before sending an acknowledgment.

8. **Checksum (16 bits):**

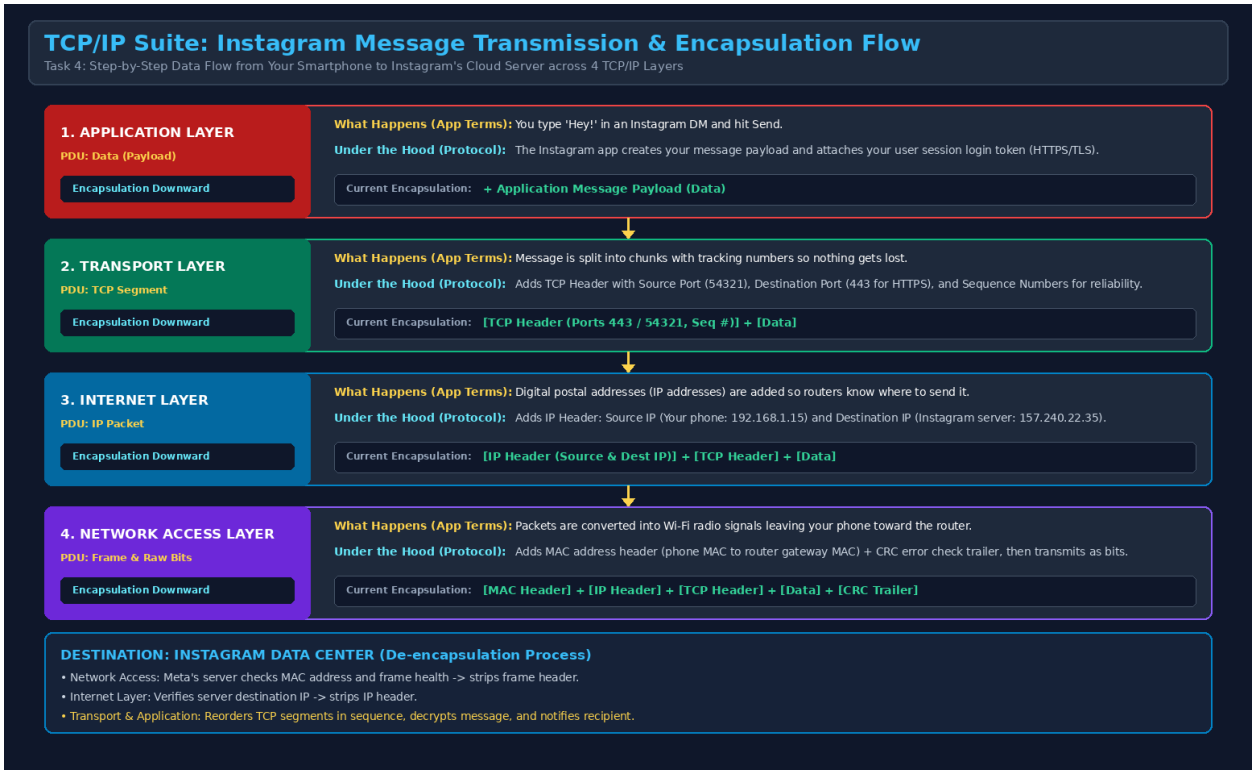
Provides error detection to verify that the TCP header and data were not corrupted during transit.

9. **Urgent Pointer (16 bits):**

Points to priority data that must be processed immediately by the receiving application.

Task 4: Step-by-Step Message Transmission Diagram (Your Device →\rightarrow→ Instagram Server)

The diagram below details the encapsulation process as a message moves down the TCP/IP stack:



Step-by-Step Explanation:

1. **Application Layer (Message Created):**

You type *"Hey!"* in an Instagram DM and hit Send. The Instagram app formats the message into an HTTPS payload with your login authentication token.

2. **Transport Layer (Message Segmented & Ports Attached):**

TCP breaks large data into manageable segments and attaches a **TCP Header** containing Source Port (54321), Destination Port (443), and Sequence Numbers to ensure reliable delivery.

3. **Internet Layer (IP Address Added):**

The segment is wrapped into an **IP Packet** by attaching your device's private IP (192.168.1.15) as the source and Meta's Instagram server IP (157.240.22.35) as the destination.

4. **Network Access Layer (Physical Transmission):**

The packet is enclosed into an **Ethernet/Wi-Fi Frame** with MAC addresses and a CRC error-check trailer. Your device's Wi-Fi card transmits the frame as 5 GHz5\text{ GHz}5 GHz radio waves over the air to the router.

Task 5: Wireshark Packet Analysis & Real-World Action

- **Captured Packet Chosen:** TCP 52140 → 443 [ACK] Seq=145 Ack=782 Win=65535 Len=0
- **TCP/IP Layer: Transport Layer**
- **How It Was Identified:**
    - Under the **Protocol** column in Wireshark, it is labeled as **TCP**.
    - Expanding the packet tree in the middle pane displays the **Transmission Control Protocol** header showing source port 52140, destination port 443 (standard encrypted web traffic), and an active ACK (Acknowledgment) flag.
- **Real-World Action in an App (e.g., Booking a Ticket on BookMyShow):**

When you confirm seat selection and click *"Proceed to Pay"* on **BookMyShow**, your device uses this TCP ACK packet to acknowledge receipt of the payment gateway's secure session data. It tells BookMyShow's server: *"I have safely received the seat lock confirmation, proceed with displaying the payment screen without dropping the session."*